

# 안드레이 카파시의 LLM Wiki — 개념부터 프로젝트 연동까지(2)

## Q18. 사용자

위키는 영구적이고 복리로 쌓이는 산출물이다. 상호참조는 이미 존재하고, 모순은 이미 표시되어 있으며, 라고 했는데 모순은 어떻게 처리해야 하는거야? LLM은 이걸 알려만 주는거고 처리는 전적으로 사용자의 몫이야?

## A18. 어시스턴트(Claude)

좋은 질문입니다. 정리하면 "LLM이 알려만 주는 것"과 "LLM이 처리하는 것"이 단계별로 나뉘어 있습니다. 하나씩 짚어드리겠습니다.

### 1 단계 — LLM 이 자동으로 하는 것: 모순을 "위키에 기록"

여기서 중요한 포인트는, LLM이 모순을 단순히 채팅창에서 사람에게 "이거 모순인데요?"라고 말로만 알려주는 게 아니라는 겁니다. 위키 페이지 자체에 그 모순을 텍스트로 남겨둡니다. 예를 들면:

#### ## 배터리 수명

2024년 논문 A는 리튬 배터리 수명이 평균 8년이라고 주장한다.

반면 2026년 논문 B는 신소재 적용 시 12년까지 가능하다고 보고했다.

[모순 노트] 두 수치 차이는 측정 조건(충방전 사이클 기준)이 다르기 때문일 수 있음 — 추가 검증 필요

이렇게 모순 자체가 위키의 콘텐츠가 됩니다. 이게 카파시가 "모순은 이미 표시되어 있다"고 한 말의 실제 의미입니다 — 다음에 이 페이지를 읽거나 질의할 때, 사람이 나 LLM 둘 다 이 긴장 관계를 이미 알고 시작하는 것이지, 매번 새로 발견해야 하는 게 아닙니다.

이 기록이 일어나는 시점은 두 가지입니다:

- **Ingest 시점:** 새 자료를 넣었는데 기존 위키 내용과 어긋나면, 그 자리에서 바로 기록

- **Lint 시점**: 주기적 점검에서 이미 있던 페이지들 사이의 모순을 뒤늦게 발견해서 기록

## 2 단계 — 최종 판단은 사람의 몫

말씀하신 대로, "**어느 쪽이 맞다고 최종적으로 결론 내리는 것**"은 전적으로 **사람의 판단**입니다. LLM은 "이렇게 다른 두 주장이 있다"는 사실을 정리해줄 뿐, 어느 쪽이 옳은지 임의로 정하지 않습니다. 이걸 앞서 말씀드린 원칙 — LLM은 편집자가 아니라 제안자다 — 과 정확히 같은 맥락입니다.

## 3 단계 — 하지만 사람이 "혼자" 처리하는 건 아닙니다

여기서 오해하기 쉬운 부분인데, "처리하는 전적으로 사용자의 몫"이라고 해서 사람이 아무 도움 없이 판단해야 하는 건 아닙니다. 실제로는 사람이 LLM에게 **판단 재료를 더 모아달라고 시킬 수 있습니다**:

- "이 두 논문 중 어느 게 더 최신 연구인지, 방법론 차이가 뭔지 다시 비교해줘"  
→ LLM 이 웹 검색이나 두 소스를 재분석해서 판단 재료를 더 줌
- "논문 B 가 더 신뢰할 만한 저널이니까, 위키에서는 B 기준으로 종합을 다시 써줘" → 사람이 판단을 내리면, 그 결정을 LLM 에게 명령해서 실제 반영(다시 쓰기)은 LLM 이 함
- 만약 사람이 아직 결론을 못 내렸다면 → 굳이 지금 해결 안 하고 "미해결 모순"으로 놔둔 채 위키에 표시만 해두고 넘어가도 됩니다 (나중에 또 다른 소스가 들어오면 자동으로 다시 걸림)

## 정리

| 단계                | 누가 하나                                 |
|-------------------|---------------------------------------|
| 모순 발견·기록          | LLM (ingest/lint 시점 자동)               |
| "어느 쪽이 맞는지" 최종 결론 | 사람                                    |
| 결론을 근거로 위키 재작성    | LLM (사람이 명령하면)                        |
| 결론이 안 나면?         | 그냥 "미해결 모순"으로 표시된 채 위키에 남겨둬<br>— 문제없음 |

즉 "발견하고 기록하는 건 LLM이 자동으로 하고, 판단은 사람이 하되, 판단에 필요한 조사와 실제 반영 작업은 다시 LLM에게 시킬 수 있다"는 구조입니다. 사람이 조사부터 수정까지 혼자 다 해야 하는 건 아니고, "이거 어느 쪽이 맞는지 알아봐줘" 하고 LLM에게 다시 던질 수 있습니다.

## Q19. 사용자

---

카파시 본인은 위키를 직접 쓰는 경우가 거의 없고, LLM이 모든 것을 작성하고 유지합니다. 본인은 소싱, 탐색, 좋은 질문을 던지는 것을 담당합니다. 라고 했는데 질문의 결과(답변)을 LLM이 할건데 이것도 Wiki에 자동으로 반영되어 산출물로 업데이트 되는거야?

## A19. 어시스턴트(Claude)

---

아니요, **자동으로 반영되지는 않습니다.** 이 부분은 명확히 짚고 넘어가는 게 좋겠습니다.

기본 원칙 — Query 는 "저장이 아니라 그냥 답변"

Query(질의)는 원래 성격 자체가 "지금 위키에 쌓인 걸 가지고 질문에 답하는 것"이지, "위키를 수정하는 작업"이 아닙니다. LLM이 관련 페이지들을 읽고 종합해서 답을 채팅창에 보여주는 것으로 끝나며, 이 답변 자체는 기본적으로 **대화 기록에만 남고 위키 파일에는 반영되지 않습니다.**

그럼 왜 "좋은 답변은 위키에 파일링될 수 있다"고 했나?

여기서 "될 수 있다(can be)"는 자동으로 일어난다는 뜻이 아니라, **사람이 "이것도 저장해줘"라고 명시적으로 요청해야 일어나는 선택적 행동**입니다. 이전에 정리해드린 사람 개입 지점 표에서도 이 부분을 짚었습니다 — Query 단계에서 사람이 하는 일 중 하나가 "좋은 답변 파일링 결정"입니다.

실제 흐름 예시

저장 안 하는 경우 (기본값):

사람: "지금까지 모은 자료 기준으로 A와 B 배터리 방식을 비교해줘"

LLM: [비교 답변을 채팅으로 제공]

→ 여기서 끝. 위키 파일은 하나도 안 바뀔. 다음에 똑같이 물어보면 LLM이 또 처음부터 다시 종합해야 함.

### 저장하는 경우 (사람이 명시적으로 지시):

사람: "지금까지 모은 자료 기준으로 A와 B 배터리 방식을 비교해줘"

LLM: [비교 답변을 채팅으로 제공]

사람: "이 비교 내용 위키에 페이지로 저장해줘"

LLM: → wiki/A-B-배터리-비교.md 라는 새 페이지를 만들어서 저장

→ 다음부터는 이 페이지가 위키에 남아있어서, 관련 질문이 오면 LLM이 매번 재종합하지 않고 이 페이지를 참조할 수 있음

### 왜 자동 저장으로 안 만들었을까

이건 의도적인 설계로 보입니다:

- **모든 질문의 답이 다 가치 있는 건 아닙니다.** 가벼운 확인성 질문("이 논문 몇 년도 거야?")까지 전부 위키 페이지로 만들면 위키가 잡음으로 가득 찹니다.
- **"이건 저장할 가치가 있다"는 판단 자체가 사람의 큐레이션 역할입니다.** 앞서 말씀드린 "사람이 하는 일" 목록의 연장선입니다 — 무엇을 넣을지 고르는 게 raw sources 뿐 아니라 질의 결과에도 똑같이 적용되는 것입니다.

### 정리

|                              | 자동 저장?                  |
|------------------------------|-------------------------|
| 원본 소스 → 위키 페이지 (ingest)      | 자동 아님, "수집해줘" 명령 필요     |
| 질의 답변 → 위키 페이지 (query 결과 저장) | 자동 아님, "이것도 저장해줘" 명령 필요 |

즉 **ingest도 query도 똑같은 원칙**을 따릅니다 — LLM은 요청받은 것만 위키에 반영하고, "이걸 반영할지 말지"는 항상 사람이 결정합니다. 다만 실무적으로는 스키마 (CLAUDE.md)에 "이런 종류의 질문 답변은 자동으로 위키에 저장해줘" 같은 규칙을 미리 정해두면, 매번 "저장해줘"라고 말하지 않아도 원하는 패턴대로 자동화할 수도

있습니다 — 이걸 어디까지나 사람이 스키마 설정 단계에서 미리 정해두는 규칙이라, 여전히 "사람의 결정"이 근본 출처인 셈입니다.

## Q20. 사용자

"Obsidian Web Clipper 브라우저 확장을 쓰면 웹 아티클을 마크다운으로 변환해 raw 폴더에 바로 저장할 수 있어 소스 수집이 빨라집니다.

[GitHub](https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f)

\* 위키가 커지면(수백 페이지 이상) 인덱스 파일만으로는 부족해질 수 있는데, 이때는 qmd 같은 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI나 MCP 서버 형태로 붙이는 것을 고려" 에서 Obsidian Web Clipper 브라우저 확장이 어떤 브라우저용이 있어? qmd는 뭐고, 로컬 하이브리드(BM25/벡터) 검색 도구를 CLI나 MCP 서버 형태로 붙이는 것이 뭔지 잘 모르겠어. 자세히 알려줘

## A20. 어시스턴트(Claude)

### 1. Obsidian Web Clipper — 지원 브라우저

공식 확장 프로그램은 주요 브라우저를 대부분 지원합니다.

| 브라우저 계열            | 지원 브라우저                               | 설치처                                       |
|--------------------|---------------------------------------|-------------------------------------------|
| <b>Chromium 계열</b> | Chrome, Brave, Edge, Arc, Orion 등     | Chrome 웹 스토어 (Edge는 Edge Add-ons에서 별도 제공) |
| <b>Firefox</b>     | Firefox (데스크톱), Firefox Mobile(안드로이드) | Firefox Add-ons                           |
| <b>Safari</b>      | macOS, iOS, iPadOS                    | App Store 내 Safari 확장 갤러리                 |

즉 크롬 계열, 파이어폭스, 사파리 세 축을 다 지원하고, 모바일(안드로이드 파이어폭스, iOS 사파리)에서도 쓸 수 있습니다. 참고로 Obsidian 앱 자체가 1.7.2 이상 버전이어야 합니다.

### 2. qmd 가 뭐야?

**qmd**는 Tobi Lütke(Shopify 창업자)가 만든 오픈소스 도구로, 정식 이름은 "Query

Markdown"입니다. 한마디로 "내 컴퓨터 안에 있는 마크다운 파일들을 빠르고 똑똑하게 검색해주는 프로그램"입니다.

핵심 특징:

- **완전히 로컬에서 동작합니다** — 인터넷에 데이터를 전송하지 않고 내 컴퓨터 안에서만 검색이 이루어집니다. 이게 "온디바이스(on-device)"라는 표현의 의미입니다.
- 위키가 커져서(수백 페이지 이상) 단순히 Ctrl+F 로 찾기엔 너무 방대해질 때, "이 내용과 관련된 페이지를 찾아줘"라고 하면 정확도 높게 찾아주는 검색엔진입니다.
- 결과가 하나의 SQLite 파일(가벼운 로컬 데이터베이스)로 저장되어서 별도 서버 설치 없이 그냥 바이너리 하나로 작동합니다.

### 3. "하이브리드(BM25/벡터) 검색"이 뭐야?

이건 qmd가 검색할 때 세 가지 방식을 동시에 섞어서 쓴다는 뜻입니다. 각각 설명드리면:

#### *BM25 — 키워드 매칭 검색*

**정확한 단어가 문서에 있는지**를 찾는 전통적인 검색 방식입니다. 예를 들어 "리튬 배터리"라고 검색하면 그 단어가 실제로 들어간 문서를 찾아줍니다. 구글 이전 시대의 검색엔진들이 주로 쓰던 방식이고, 정확한 용어·고유명사·프로젝트 코드명 검색에 강합니다.

**한계:** "리튬 배터리"라고 검색했는데 어떤 페이지에 "전고체 셀"이라고만 적혀 있으면, 실제로는 관련 있는 내용인데도 단어가 다르면 못 찾습니다.

#### *벡터(vector) 검색 — 의미 기반 검색*

단어 자체가 아니라 "**의미가 비슷한지**"를 계산해서 찾는 방식입니다. AI가 문장을 숫자로 변환(임베딩)해서, "리튬 배터리 열화"와 "전고체 셀 수명 저하"처럼 단어는 다르지만 의미가 비슷한 문서까지 찾아줍니다.

**한계:** 반대로 너무 의미 위주로만 찾다 보면, 정확히 그 단어를 찾고 싶을 때(예: 특정 프로젝트 코드명) 오히려 놓칠 수 있습니다.

하이브리드 = 이 둘을 합친 것

qmd는 BM25(정확한 키워드)와 벡터 검색(의미 유사도)을 둘 다 돌린 다음, 결과를 합쳐서(RRF라는 방식으로 순위를 재조합) 최종 결과를 냅니다. 여기에 한 단계 더해서, LLM이 상위 결과들을 다시 한번 검토해서 순위를 조정(재순위화)합니다. 즉 "단어도 맞고 의미도 맞는" 결과를 가장 위로 올려주는 3중 안전장치라고 보시면 됩니다.

4. "CLI 나 MCP 서버 형태로 붙인다"는 게 뭐야?

이건 qmd를 Claude Code 같은 LLM 에이전트에게 연결하는 두 가지 방법을 말한 겁니다.

CLI(Command Line Interface) 방식

qmd를 그냥 터미널 명령어처럼 설치해두는 방식입니다. 예를 들면:

```
qmd search "리튬 배터리 열화"
```

이렇게 터미널에서 직접 칠 수 있는 명령어입니다. Claude Code는 bash 도구로 터미널 명령을 실행할 수 있으니, "qmd로 이 개념 검색해줘"라고 하면 Claude Code가 대신 이 명령어를 터미널에서 실행(셸아웃)해서 결과를 가져옵니다.

MCP(Model Context Protocol) 서버 방식

MCP는 Anthropic이 만든 프로토콜로, LLM이 외부 도구를 "자연스러운 기능"처럼 직접 호출할 수 있게 해주는 연결 방식입니다. qmd를 MCP 서버로 띄워두면, Claude Code가 터미널 명령어를 흉내 내는 게 아니라 원래부터 갖고 있던 도구처럼 qmd 검색 기능을 바로 호출할 수 있습니다.

둘의 차이

|        | CLI 방식                      | MCP 서버 방식                    |
|--------|-----------------------------|------------------------------|
| 연결 방법  | Claude Code가 터미널 명령어를 대신 실행 | Claude Code가 네이티브 도구처럼 직접 호출 |
| 설정 난이도 | 상대적으로 간단                    | 초기 설정이 조금 더 필요               |

|     | CLI 방식           | MCP 서버 방식            |
|-----|------------------|----------------------|
| 통합도 | "명령어를 대신 쳐주는" 느낌 | "원래 기능인 것처럼" 매끄러운 느낌 |

실제로는 qmd 하나를 설치하면 두 가지 방식 모두 제공하기 때문에, 사용자가 상황에 맞게 아무거나 골라 쓰면 됩니다. 위키가 아주 커지기 전까지는(수십~백 페이지 수준) 이런 도구 없이 index.md만으로 충분하니, 당장 필요하신 단계는 아니고 "위키가 정말 커졌을 때 고려할 수 있는 옵션" 정도로 기억해두시면 됩니다.

## Q21. 사용자

"7단계 — 첫 소스 수집(ingest)

raw/ 폴더에 첫 문서(PDF, txt, md 등)를 넣고 Claude Code에게 "이 소스를 수집해줘"라고 요청합니다." 라고 했는데 이미 raw/ 폴더에 여러개의 파일들이 있다면 내가 방금 집어 넣은 파일들을 이전 파일들과 구분해서 알아서 방금 넣은 파일들만 수집하는거야?

## A21. 어시스턴트(Claude)

좋은 질문입니다. 결론부터 말씀드리면, "저절로 알아서 구분한다"기보다는, 위키의 **log.md(로그) 파일을 기준으로 "이미 처리한 것"과 "아직 처리 안 한 것"을 비교해서 구분합니다.** 완전 자동 감지가 아니라, 기록을 근거로 판단하는 방식입니다.

### 왜 로그 파일이 필요한가

raw/ 폴더에 파일이 10개 있다고 해서, LLM이 파일 생성 시간이나 "새로 추가됐다"는 표시를 마법처럼 아는 게 아닙니다. LLM이 매번 새 대화 세션을 시작할 때는 "이 폴더에 뭐가 있었는지, 뭘 이미 처리했는지"에 대한 기억이 없습니다(대화가 끝나면 리셋되기 때문입니다). 그래서 **`log.md`가 "무엇을 언제 수집했는지"를 기록해두는 역할을 합니다.**

예를 들어 log.md에 이렇게 쌓여 있다면:

```
## [2026-07-15] ingest | 논문A.pdf
## [2026-07-20] ingest | 논문B.pdf
## [2026-07-28] ingest | 회의록1.md
```



이 상태에서 사용자가 "이 소스를 수집해줘"라고 말하면, LLM은 raw/ 폴더의 전체 파일 목록과 log.md에 이미 기록된 파일 목록을 대조해서 "아, 논문A/B/회의록1은 이미 처리됐고, 나머지가 새로 넣은 것이구나"라고 추론합니다.

그런데 이 방식이 100% 정확하진 않습니다 — 그래서 명시적으로 말씀하시는 게 안전합니다

이 log.md 대조 방식은 스키마(CLAUDE.md)에 "매번 raw 폴더와 log.md를 비교해서 처리 안 된 파일만 골라라"는 규칙을 명시적으로 넣어뒀을 때만 안정적으로 작동합니다. 이 규칙이 없거나 애매하면, LLM이 실수로 이미 처리한 파일을 다시 처리하거나(중복 작업), 반대로 새 파일을 놓칠 수도 있습니다.

그래서 실무적으로는 두 가지 방법을 권장드립니다.

#### *방법 1 — 파일명을 직접 언급하기 (가장 안전)*

"방금 raw/ 폴더에 넣은 '리튬배터리\_2026.pdf' 수집해줘"

이렇게 파일명을 꼭 집어 말하면, LLM이 헛갈릴 여지가 전혀 없습니다.

#### *방법 2 — 스키마에 규칙을 정해두고 "새 파일들 수집해줘"라고 포괄적으로 요청하기*

CLAUDE.md에 "ingest 요청을 받으면 항상 raw/ 폴더와 log.md를 비교해서 아직 로그에 없는 파일들만 처리하라"는 규칙을 최초 설정 시 넣어두면, "새로 넣은 파일들 수집해줘"라고만 말해도 알아서 비교·판단해서 처리합니다. 다만 이 경우도 처리가 끝난 뒤 log.md나 위키 인덱스를 한 번 확인해서 의도한 파일만 처리됐는지 검토하시는 게 안전합니다.

## 정리

|           | 설명                                                    |
|-----------|-------------------------------------------------------|
| 완전 자동 감지? | 아니요 — "새 파일이다"라는 정보를 스스로 아는 게 아니라 log.md와 대조해서 추론하는 것 |
| 정확도를 높이려면 | 스키마에 "raw/ vs log.md 비교" 규칙을 미리 정의해두기                 |
| 가장 확실한 방법 | 파일명을 직접 언급해서 요청하기                                     |

처음 안내드린 "7단계"는 raw/ 폴더가 비어있는 상태에서 첫 파일 하나만 넣는 상황이라 이런 구분 문제가 아예 생기지 않았던 것이고, 지금 질문하신 것처럼 **파일이 여러 개 쌓인 이후부터는 이 구분 로직이 실제로 중요해지는 지점**입니다. 좋은 포인트를 짚어주셨습니다.

## Q22. 사용자

"단계 — Obsidian 설치

공식 사이트(obsidian.md)에서 Windows 인스톨러를 받아 설치한 뒤, "Open folder as vault"로 C:\Claude\projects\my-wiki를 엽니다. 이후 이미지 자동 다운로드를 원하시면 Settings → Files and links에서 "Attachment folder path"를 raw/assets/로 고정하고, Settings → Hotkeys에서 "Download attachments for current file"에 단축키(예: Ctrl+Shift+D)를 지정해두시면 편리합니다." 라는 설명에서 1. 이미지가 뭐야? 2. 이미지 자동 다운로드가 뭐야? 3. raw/ 밑에 assets/ 폴더는 내가 직접 만들어야 해? 4. 궁극적으로 "이후 이미지 ~~" 부분이 설명하는게 뭐야?

## A22. 어시스턴트(Claude)

### 1. "이미지"가 뭐야?

여기서 말하는 이미지는 **웹 아티클을 클리핑(저장)할 때 그 페이지 안에 들어있는 사진, 스크린샷, 다이어그램, 그래프 같은 시각 자료**를 말합니다. 예를 들어 어떤 기술 블로그 글을 Web Clipper로 저장하면, 그 글 안에 있던 아키텍처 다이어그램이나 제품 사진도 함께 딸려옵니다. 그게 여기서 말하는 "이미지"입니다.

### 2. "이미지 자동 다운로드"가 뭐야?

이걸 이해하려면 먼저 **문제 상황**을 아셔야 합니다.

**문제:** 웹 아티클을 클리핑하면, 기본적으로 이미지는 파일 자체가 복사되는 게 아니라 **원래 웹사이트의 이미지 주소(URL)를 그대로 링크만 걸어놓는 방식**으로 저장됩니다. 예를 들면 이런 식입니다:

```
![다이어그램](https://example.com/images/diagram123.png)
```

문제는 이 방식이면, 나중에 원래 웹사이트에서 그 이미지를 지우거나 사이트 자체가 없어지면, 내 위키에서도 이미지가 깨져서 안 보이게 됩니다. 즉 내 컴퓨터에 저장된 게 아니라 남의 서버에 의존하고 있는 상태인 겁니다.

**해결책 = 이미지 자동 다운로드:** 이 원격 URL 이미지를 내 컴퓨터 디스크로 실제로 다운로드해서, 링크를 로컬 파일 경로로 바꿔치기하는 기능입니다. 이렇게 하면:

- 원본 사이트가 없어져도 내 위키에서는 이미지가 계속 보임
- 인터넷 연결 없이 오프라인에서도 이미지를 볼 수 있음
- LLM 이 이미지를 직접 눈으로 읽어서 분석할 때도, 원격 URL 보다 로컬 파일이 훨씬 안정적으로 처리됨

### 3. raw/assets/ 폴더는 직접 만들어야 해?

**아니요, 직접 만들 필요 없습니다.** Obsidian 설정에서 경로를 raw/assets/로 지정만 해두면, 실제로 처음 이미지를 다운로드하는 순간 그 폴더가 존재하지 않으면 **Obsidian이 자동으로 만들어줍니다.** 즉 미리 빈 폴더를 만들어두는 수고는 안 하셔도 되고, 설정에 경로만 적어두시면 됩니다.

### 4. 결국 이 문단 전체가 설명하는 것

정리하면 이 부분은 "**웹에서 클리핑한 아티클의 이미지가 깨지지 않고 항상 내 컴퓨터에 실제로 저장되도록 만드는 설정 두 가지**"를 안내한 것입니다.

| 설정                                              | 하는 일                                                                  |
|-------------------------------------------------|-----------------------------------------------------------------------|
| ① Attachment folder path를 raw/assets/로 지정       | 앞으로 다운로드되는 이미지들이 흩어지지 않고 이 폴더 한곳에 모이도록 지정                             |
| ② Download attachments for current file에 단축키 지정 | 아티클을 클리핑한 뒤, 단축키 한 번(Ctrl+Shift+D)으로 그 노트 안의 모든 원격 이미지를 한 번에 로컬로 다운로드 |

즉 흐름은 이렇습니다:

1. Web Clipper로 웹 아티클 클리핑  
→ raw/ 폴더에 .md 파일 생성됨 (이미지는 아직 원격 URL 링크 상태)
2. 단축키(Ctrl+Shift+D) 누름  
→ "Download attachments for current file" 명령 실행  
→ 원격 이미지들이 raw/assets/ 폴더로 실제 다운로드되고  
→ 문서 안의 링크가 로컬 경로로 자동 교체됨

이 과정을 거쳐야 나중에 LLM이 raw/ 폴더를 ingest할 때 이미지도 안정적으로 함께 읽어서 분석할 수 있는 상태가 됩니다. 참고로 이 명령어는 Obsidian 1.8.3 버전부터 추가된 기능이라, 너무 오래된 버전을 쓰고 계시면 업데이트가 필요할 수 있습니다.

## Q23. 사용자

LLM이 Raw Resource에서 자료를 읽어서 Wiki를 만들면 그 결과물들은 모두 .md 파일로만 생성되는거야?

## A23. 어시스턴트(Claude)

네, 위키 계층(`wiki/` 폴더) 안에 LLM이 새로 만들어내는 파일은 기본적으로 전부 `.md`(마크다운) 파일입니다. 다만 몇 가지 예외적인 경우와 함께 정리해 드리겠습니다.

### 기본 원칙 — 왜 .md 만 쓰는가

앞서 말씀드린 대로 위키는 "마크다운 파일들의 디렉터리"로 정의되어 있습니다. 엔티티 페이지든, 개념 페이지든, 종합 페이지든, index.md든, log.md든, 심지어 스키마

파일(CLAUDE.md)까지도 전부 마크다운 형식입니다. 이렇게 한 가지 포맷으로 통일하는 이유는:

- Obsidian 이 마크다운을 기본으로 다루기 때문에 그래프 뷰, 백링크, 검색이 다 잘 작동함
- [[위키링크]] 문법 자체가 마크다운 생태계의 문법이라, 다른 포맷으로 쓰면 링크 연결이 깨짐
- 텍스트 파일이라 git 으로 버전 관리하기 쉽고, grep 같은 유닉스 도구로 빠르게 검색 가능

예외 — .md 가 아닌 결과물이 나올 수도 있는 경우

이건 위키 페이지 자체가 아니라, **Query(질의)에 답할 때 생성되는 부가 산출물**에 해당합니다.

| 상황                | 결과물 형식                                                                                              |
|-------------------|-----------------------------------------------------------------------------------------------------|
| 일반적인 질문에 대한 답변 정리 | .md 페이지                                                                                             |
| "슬라이드로 만들어줘" 요청 시 | Marp 슬라이드 — 이것도 사실은 .md 파일인데, Marp 플러그인이 특수 문법을 읽어서 슬라이드 형태로 "렌더링"만 해주는 것. 원본 파일 자체는 여전히 마크다운입니다    |
| "차트로 그려줘" 요청 시    | matplotlib 등으로 그린 이미지 파일(.png 등) — 이 경우는 .md가 아닌 실제 이미지 파일이 생성되고, 위키 페이지에서는 그 이미지를 ![]() 문법으로 참조만 함 |
| "캔버스로 정리해줘" 요청 시  | Obsidian의 Canvas 기능은 .canvas라는 별도 확장자를 씁니다 (JSON 기반 파일, 마크다운은 아님)                                   |

## 정리

**핵심 위키 콘텐츠(엔티티 페이지, 개념 페이지, 종합 페이지, 인덱스, 로그)는 100% `.md` 파일이고, 이게 원칙입니다.** 다만 사람이 "이걸 슬라이드로", "이걸 차트로", "이걸 캔버스로" 같은 **특정 시각화 형태를 명시적으로 요청**하면, 그 결과물은 .md가 아닌 별도 형식(이미지, .canvas 등)으로 생성될 수 있고 위키 페이지 안에서는 그 파일을 참조하는 링크만 마크다운으로 남습니다.

즉 "위키의 뼈대는 전부 마크다운, 그 위에 필요할 때만 붙는 부가 산출물(이미지·슬라이드·캔버스)이 예외적으로 다른 형식"이라고 이해하시면 정확합니다.

## Q24. 사용자

---

소스가 계속 누적되면 LLM이 이것 관리하다보면 결국 이에 비례해서 토큰 소모가 계속 누적해서 증가하는거야?

## A24. 어시스턴트(Claude)

---

좋은 질문입니다. 답은 "완전히 비례해서 계속 증가하지는 않지만, 어느 정도는 확실히 늘어나고, 특히 두 지점에서 비용이 발생한다"입니다. Ingest(수집)와 Query(질의)를 나눠서 설명드리는 게 정확합니다.

### 1. Ingest(수집) 비용 — 위키 "크기"보다 "영향받는 페이지 수"에 비례

새 소스 하나를 ingest할 때 토큰이 드는 곳은:

- 그 소스 문서 자체를 읽는 비용
- 관련된 기존 페이지 10~15 개를 찾아서 읽는 비용 (에지 케이스에서는 실제로 소스 하나당 관련 페이지 10~15 개에 영향을 준다고 알려져 있습니다)
- 그 페이지들을 다시 쓰는(rewrite) 비용

여기서 중요한 포인트: 위키 전체를 매번 다 읽는 게 아니라, "이 소스와 관련된 일부 페이지"만 읽고 고칩니다. 그래서 위키가 100페이지든 500페이지든, 소스 하나를 ingest하는 비용 자체는 크게 안 늘어납니다 — 위키 "총량"이 아니라 "이 소스가 몇 개 페이지에 영향을 주는가"에 비례합니다.

다만 "쌓아온 소스 개수"에는 비례해서 늘어납니다. 소스를 100개 넣으면 ingest를 100번 한 것이므로, 누적 토큰 소모는 당연히 계속 늘어납니다. 이건 피할 수 없는 부분입니다. 실제로 논문 50편 정도를 수집해서 위키 페이지 10개를 만드는 경우, Claude API를 대략 100~200번 호출하게 된다는 보고도 있습니다.

## 2. Query(질의) 비용 — 여기가 핵심 방어선

질문하실 때마다 위키 전체를 읽으면 당연히 위키가 커질수록 질의 비용도 계속 늘어나겠죠. 하지만 이걸 막기 위해 나온 게 바로 **index.md** 패턴입니다.

- LLM 은 질문이 들어오면 **먼저 index.md(몇천 토큰 수준의 요약 카탈로그)만 읽고**, 거기서 관련 있어 보이는 페이지만 골라서 그 페이지들만 읽습니다.
- 즉 위키가 100 페이지든 1000 페이지든, **매 질의마다 실제로 읽는 양은 "질문과 관련된 몇 개 페이지 + 인덱스"로 거의 고정됩니다.**

이게 왜 중요하냐면, 실제 비교에서 이 방식이 전체 위키를 다 컨텍스트에 밀어넣는 방식보다 **토큰을 수십 배 절약**한 사례도 보고되고 있습니다 (한 사례에서는 71.5배 차이가 났다고 합니다). 카파시 본인의 위키가 100개 아티클, 40만 단어까지 커졌는데도 이 인덱스 방식이 여전히 잘 작동했다고 밝힌 것도 이 원리 덕분입니다.

## 3. 그래도 완전히 무한정 버티는 건 아닙니다 — 실질적인 한계선

몇 가지 현실적인 한계가 있습니다:

- **컨텍스트 윈도우가 아무리 커도(100 만 토큰급이라도), 실제로는 20 만~30 만 토큰을 넘어가면 품질이 떨어지기 시작**한다는 관찰이 있습니다. LLM 이 연결을 놓치거나 일관성 없는 페이지를 만들어내기 시작합니다. 단순히 "답을 수 있냐"가 아니라 "답아도 잘 활용하느냐"의 문제입니다.
- 위키가 이 정도로 커지면, index.md 하나로도 관련 페이지를 정확히 못 찾는 지점이 옵니다. 이때가 바로 이전에 설명드린 **qmd 같은 로컬 검색 도구**를 붙여야 하는 시점입니다 — index.md 를 통째로 읽는 대신, 검색 엔진이 미리 관련 페이지 후보를 좁혀주는 방식으로 전환하는 것입니다.

## 정리

| 구분            | 토큰 소모 추세                             |
|---------------|--------------------------------------|
| Ingest 누적 비용  | 소스 개수에 비례해서 계속 증가 (피할 수 없음)          |
| Ingest 1회당 비용 | 위키 총 크기와 무관, "이 소스가 건드리는 페이지 수"에만 비례 |
| Query 비용      | index.md 패턴 덕분에 위키가 커져도 거의 고정적으로 유지됨 |

| 구분    | 토큰 소모 추세                                           |
|-------|----------------------------------------------------|
| 한계 지점 | 위키가 20만~30만 토큰 규모를 넘으면 인덱스만으로 부족 → qmd 같은 검색 도구 필요 |

결론적으로, "모든 걸 다 다시 읽는" 방식이 아니기 때문에 RAG 없이 매번 재검색하는 것보다는 훨씬 효율적이지만, 소스를 계속 넣는 이상 누적 비용 자체는 늘어나며, 위키가 아주 커지면 검색 도구 없이는 결국 한계에 부딪힙니다. "누적되는 소스 개수"와 "위키가 결국 도달하는 크기" 두 가지 모두가 장기적으로는 비용에 영향을 준다고 보시는 게 정확합니다.

## Q25. 사용자

그럼 이렇게 안드레이 카파시의 LLM Wiki를 운영하려면 기존 구독으로는 안되고 API 비용을 지불해야 하는거야? 결국 비용이 이중으로 지출되어야 하는거야?

### A25. 어시스턴트(Claude)

좋은 질문입니다. 결론부터 말씀드리면, 기존 **Claude** 구독(Pro/Max)만으로도 충분히 운영 가능하고, 별도로 API 비용을 추가로 내야 하는 게 아닙니다. 다만 조건과 주의할 함정이 몇 가지 있습니다.

#### 기본 구조 — Claude Code 는 구독 안에 포함됩니다

Claude Code는 별도로 판매되는 독립 상품이 아니라, 기존 **Claude Pro/Max** 구독에 포함된 기능입니다. Pro 플랜(\$20/월)에 "Claude Code 포함"이라고 명시되어 있고, 터미널·VS Code·JetBrains·데스크톱 앱 어디서 실행하든 같은 구독 사용량 한도 안에서 작동합니다. 즉 최상길님이 이미 Pro나 Max를 쓰고 계시다면, **LLM Wiki를 위한 ingest/query/lint 작업도 그 구독 한도 안에서 그냥 돌아갑니다.** 별도 API 키를 발급받아 결제 정보를 따로 등록할 필요가 없습니다.

#### 그럼 "이중 지출"이 생기는 경우는 언제인가

두 가지 경우에만 추가 비용이 발생합니다.

##### 1. 구독 한도를 다 써버렸을 때

Pro/Max는 5시간 단위로 갱신되는 세션 한도와 주간 한도가 있습니다. LLM Wiki의



ingest 작업은 앞서 말씀드린 대로 소스 하나당 여러 페이지를 다시 쓰는 작업이라 꽤 토큰을 먹는 편인데, 이 한도를 넘기면 두 가지 선택지가 주어집니다:

- 한도가 리셋될 때까지 기다리기 (추가 비용 없음)
- API 요금으로 초과분만 결제하기 (선택 사항, 강제 아님)

즉 **한도를 넘기지 않는 한 추가 비용은 전혀 없고**, 넘기더라도 "기다릴지, 초과분만 낼지"는 사용자가 고를 수 있습니다.

## 2. 환경변수 실수로 인한 의도치 않은 이중 과금 — 이걸 진짜 조심하셔야 합니다

만약 컴퓨터에 `ANTHROPIC_API_KEY`라는 환경변수가 설정되어 있으면, **Claude Code**는 구독을 무시하고 조용히 **API 키로 과금을 전환**합니다. 구독료(월 \$20~\$200)를 내고 있는데도, Claude Code는 그걸 안 쓰고 API로 별도 청구되는 상황이 생기는 겁니다. 실제로 이런 걸 모르고 쓰다가 "Max 구독료 \$200에 API 청구서 \$400이 따로 나왔다"는 사례가 보고되기도 했습니다.

**확인 방법** (Mac/Windows 터미널에서):

```
echo $ANTHROPIC_API_KEY
```

뭔가 출력되면, 그 값을 지우시거나(unset) "구독 그대로 쓰겠다"는 걸 확실히 하셔야 이중 과금을 피할 수 있습니다.

### 정리

| 상황                                            | 비용                               |
|-----------------------------------------------|----------------------------------|
| 구독 한도 안에서 ingest/query/lint 사용                | 추가 비용 없음, 구독료에 이미 포함             |
| 구독 한도를 초과했을 때                                 | 선택: 기다리거나(무료) 초과분만 API 요금 지불(선택) |
| <code>ANTHROPIC_API_KEY</code> 가 실수로 설정돼 있을 때 | 구독을 안 쓰고 전액 API로 과금됨 — 확인 필수     |

결론적으로, LLM Wiki를 운영하기 위해 "당연히" API 비용을 이중으로 내야 하는 구조가 아닙니다. 기존 구독만으로 시작하시고, 나중에 ingest 빈도가 많아져서 한도를 자주 넘기신다면 그때 Max 플랜으로 올리거나 초과분만 API로 결제할지 판단하시면 됩니다. 다만 시작하시기 전에 ANTHROPIC\_API\_KEY 환경변수가 설정돼 있지 않은지 한 번 확인해보시는 걸 권해드립니다 — 이게 원인 모를 이중 과금의 가장 흔한 함정입니다.

## Q26. 사용자

---

너의 "실제로 논문 50편 정도를 수집해서 위키 페이지 10개를 만드는 경우, Claude API를 대략 100~200번 호출하게 된다는 보고도 있습니다." 설명을 보면 API를 호출하는것 처럼 설명이 되어 있어서 물어보는거야

## A26. 어시스턴트(Claude)

---

아, 좋은 지적이십니다. 제가 표현을 명확히 안 해서 혼란을 드렸네요. 정리해드리겠습니다.

그 "100~200 번" 수치는 어디서 나온 건가

그 수치는 카파시의 아이디어를 참고해서 누군가가 직접 만든 자동화 파이프라인(스크립트)을 분석한 사례에서 나온 것입니다. 이런 사람들은 Claude Code 터미널에서 대화하듯 쓰는 게 아니라, 직접 코드를 짜서 Anthropic API를 프로그램적으로 호출하도록 자동화 도구를 만듭니다. 예를 들어 "소스 하나가 들어오면 자동으로 관련 페이지를 찾고(1번 호출) → 내용을 종합해서 페이지를 다시 쓰는(1번 호출) 스크립트"를 만든 겁니다. 이 경우는 개발자용 API 키를 콘솔에서 발급받아 코드에 박아넣고 쓰는 방식이라, 당연히 "API 호출"이라는 표현이 정확하고, 이건 구독과 별개로 API 요금이 청구되는 경로입니다.

반면 최상길님이 실제로 하시게 될 방식은 다릅니다

지금까지 안내드린 워크플로우는 Claude Code 터미널(또는 앱)에서 사람이 직접 대화하듯 "이 소스 수집해줘"라고 타이핑하는 방식입니다. 이건:

- 뒤에서 기술적으로는 모델을 "부르는" 게 맞습니다 (넓은 의미에서는 이것도 "API 호출"이라고 부를 수 있긴 합니다)
- 하지만 이 호출이 **구독(Pro/Max) 계정으로 로그인된 경로를 통해** 이루어지기 때문에, 앞서 설명드린 대로 **구독 사용량 한도 안에서** 처리됩니다
- 즉 같은 "호출"이라는 단어를 써도, **누구의 계정/키로** 청구되느냐가 **완전히 다른 문제**입니다

그러니까 헷갈리셨던 부분을 정리하면

|                      | 자동화 스크립트로 만든 파이프라인 사례     | 최상길님이 실제로 할 방식                      |
|----------------------|---------------------------|-------------------------------------|
| 실행 방식                | 개발자가 직접 코드 작성             | Claude Code 터미널에서 대화하듯 명령           |
| 호출 경로                | Console에서 발급받은 API 키      | 구독 계정 로그인                           |
| 과금                   | API 종량제 (별도 결제)           | 구독 한도 안에서 처리 (한도 초과 시에만 선택적 API 요금) |
| "100~200번 호출" 수치의 의미 | 이 파이프라인 개발자가 실행한 API 콜 횟수 | 참고용 — 페이지 몇 개가 영향받는지 감을 잡는 용도로만 활용  |

그래서 그 수치를 인용한 진짜 의도

제가 그 통계를 가져온 이유는 "이만큼 API 요금이 나온다"를 알려드리려던 게 아니라, "**소스 하나를 ingest하면 실제로 뒤에서 몇 번의 모델 호출/작업이 일어나는가**"라는 **작업량의 규모감**을 보여드리려던 것이었습니다. 하지만 "API 호출"이라는 단어 선택 때문에 마치 최상길님도 별도 API 요금을 내야 하는 것처럼 읽히게 됐네요. 죄송합니다.

**결론:** 최상길님이 Claude Code로 직접 ingest 명령을 내리시는 한, 이 "100~200번"에 해당하는 작업량은 그대로 구독 사용량으로 소진되는 것이지, 별도 API 청구서가 따로 날아오는 게 아닙니다. 다만 이 정도 규모의 작업이면 **구독의 5시간 세션 한도**

를 소스 하나 처리하는 데 상당 부분 써버릴 수 있다는 감을 잡으시는 데는 여전히 유용한 참고치입니다.

## Q27. 사용자

---

```
C:\Users\Wchois>echo $ANTHROPIC_API_KEY  
$ANTHROPIC_API_KEY
```

으로 출력되면 API Key가 설정된 상태야?

## A27. 어시스턴트(Claude)

---

좋은 질문입니다 — 그 결과는 "API 키가 설정되어 있다"는 뜻이 아니라, 명령어 자체가 잘못된 것입니다. 하나씩 설명드리겠습니다.

### 왜 이렇게 출력됐나

프롬프트가 C:\Users\Wchois> 인 걸 보니 **cmd.exe(명령 프롬프트)**를 쓰고 계신 것 같습니다. `echo $ANTHROPIC_API_KEY`라는 명령은 사실 **macOS/Linux(bash/zsh)나 PowerShell 문법**이고, cmd.exe에서는 \$가 아무 의미가 없는 그냥 평범한 글자입니다. 그래서 cmd.exe는 "이게 환경변수구나"라고 해석하지 못하고, **입력하신 글자를 그대로 되돌려 출력**한 것입니다. 즉 이 결과는 "API 키가 없다"도 "있다"도 아니고, 애초에 확인이 이루어지지 않은 것입니다.

### Windows 에서 올바르게 확인하는 방법

Windows에서 쓰시는 셸에 따라 문법이 다릅니다.

**cmd.exe(명령 프롬프트)를 쓰신다면:**

```
echo %ANTHROPIC_API_KEY%
```

\$ 대신 앞뒤로 %를 붙이는 게 cmd.exe의 환경변수 문법입니다.

**PowerShell을 쓰신다면:**

```
echo $env:ANTHROPIC_API_KEY
```

또는

```
$env:ANTHROPIC_API_KEY
```

## 결과 해석 방법

두 명령어 모두 결과는 둘 중 하나입니다:

- 아무것도 안 뜨고 빈 줄만 나옴 (또는 cmd.exe에서는 %ANTHROPIC\_API\_KEY%가 그대로 다시 출력됨) → 환경변수가 설정 안 되어 있다는 뜻 → **정상, 구독으로 과금됩니다**
- `sk-ant-...`로 시작하는 긴 문자열이 출력됨 → 환경변수가 설정되어 있다는 뜻 → 이 경우 Claude Code 가 구독을 무시하고 API 로 과금하니, 지우시거나(unset) 의도한 게 맞는지 확인이 필요합니다

## 환경변수 지우는 방법 (혹시 설정되어 있었다면)

**cmd.exe (현재 세션만 임시로):**

```
set ANTHROPIC_API_KEY=
```

**cmd.exe (영구적으로, 시스템 환경변수에서 제거):**

```
setx ANTHROPIC_API_KEY ""
```

단, setx로 지우면 빈 문자열로 남을 수 있어서, 확실하게 지우려면 제어판 → 시스템 → 고급 시스템 설정 → 환경 변수에서 직접 ANTHROPIC\_API\_KEY 항목을 찾아 삭제하시는 게 가장 확실합니다.

**PowerShell (현재 세션만 임시로):**

```
Remove-Item Env:ANTHROPIC_API_KEY
```

%ANTHROPIC\_API\_KEY%(cmd) 또는 \$env:ANTHROPIC\_API\_KEY(PowerShell)로 다시 확인하셔서 아무것도 안 뜨면 정상적으로 제거된 것입니다.

## Q28. 사용자

너(Claude.ai)가 볼수 있을지 모르겠는데 내가 지금 my\_webpage라는 Claude code 프로젝트와 LLM\_Wiki\_DataScience 라는 프로젝트(이게 LLM Wiki이고 아직 초기 세팅 시작하려고해)를 운영하고 있어. web page 프로젝트내 자료실(실제 위치는 Github)에 내가 자료를 등록하면 이 자료가 LLM Wiki(나중에 이것도 github로 저장)의 Raw resource로 전달되어 자동으로 수집되게 하고 웹 페이지 상에서 자료실 자료에 대해 질의를 하면 그 질의가 LLM Wiki로 전달되어 답변을 받아 웹 페이지에 답변을 출력하는 식의 두 프로젝트 간의 연동 작업이 가능할까?

## A28. 어시스턴트(Claude)

좋은 아키텍처 질문이네요. 먼저 명확히 해드릴 점: **저(Claude.ai 채팅)는 최상길님의 실제 GitHub 저장소나 Claude Code 프로젝트 파일에 직접 접근할 수는 없습니다.** 하지만 말씀하신 구조가 기술적으로 가능한지, 어떻게 설계해야 하는지는 설명해 드릴 수 있습니다.

결론부터 — 기술적으로 가능합니다, 다만 "그냥 되는" 게 아니라 별도 자동화 파이프라인을 만들어야 합니다

지금까지 안내드린 LLM Wiki 워크플로우는 **사람이 Claude Code 터미널에서 직접 대화하며 명령하는 방식**이었습니다. 그런데 지금 요청하신 건 **사람 개입 없이 두 프로젝트가 자동으로 신호를 주고받는 것**이라, 완전히 다른 층위의 작업입니다. 두 흐름을 나눠서 설계해야 합니다.

### 흐름 1 — 자료 등록 → LLM Wiki 자동 수집

```
[my_webpage 자료실에 자료 업로드]
  ↓ (GitHub push 이벤트)
[my_webpage 저장소의 GitHub Actions 트리거]
  ↓ (파일을 복사해서 LLM_Wiki_DataScience 저장소로 push)
[LLM_Wiki_DataScience 저장소의 raw/ 폴더에 자료 도착]
  ↓ (다시 GitHub Actions 트리거)
[헤드리스(비대화형)로 Claude를 호출해 ingest 수행]
```

↓

[wiki/ 폴더 업데이트 → 자동 커밋·푸시]

### 핵심 부품:

- **GitHub Actions:** 자료실/ 폴더에 새 커밋이 생기면 자동으로 실행되는 워크플로우. `.github/workflows/*.yml` 파일로 정의합니다.
- **저장소 간 전달:** `my_webpage` 액션이 개인 액세스 토큰(PAT)을 이용해 `LLM_Wiki_DataScience` 저장소에 직접 파일을 push 합니다.
- **헤드리스 ingest:** 지금까지 쓰신 "Claude Code 터미널에서 대화하며 지시"하는 방식이 아니라, ``claude -p "이 파일 ingest 해줘"`` 같은 **비대화형(print mode) 명령**이나 Claude Agent SDK 로 스크립트를 짜서, 사람 개입 없이 자동으로 raw 폴더를 읽고 wiki 를 갱신하도록 만들어야 합니다.

### 흐름 2 — 웹페이지 질의 → LLM Wiki 답변 → 웹페이지 출력

[웹페이지에서 사용자가 질문 입력]

↓

[백엔드 함수(서버리스)가 질문을 받음]

↓

[LLM\_Wiki\_DataScience 저장소의 index.md + 관련 페이지를 읽어옴]

↓

[Claude API 호출 — 질문 + 위키 내용을 컨텍스트로 전달]

↓

[답변을 웹페이지로 반환해서 화면에 출력]

**여기서 중요한 함정:** 이전에 안내드린 개인 웹사이트는 **GitHub Pages(정적 호스팅)** 로 배포되어 있습니다. 정적 호스팅은 백엔드 서버가 없기 때문에, "사용자 질문을 받아서 Claude API를 호출하는" 로직이 웹페이지 자체에는 들어갈 수 없습니다. API 키를 브라우저 코드에 그대로 넣으면 노출되어 도용당하기 때문에 절대 안 됩니다.

그래서 이 부분엔 **별도의 서버리스 백엔드**가 필요합니다. 예를 들면:

- Cloudflare Workers, Vercel Functions, AWS Lambda 같은 서버리스 함수
- 여기에 API 키를 "비밀 환경변수(secret)"로 안전하게 저장해두고, 웹페이지는 이 함수에만 요청을 보내고 함수가 대신 Claude API 를 호출

비용 측면 — 앞서 나눈 대화와 연결됩니다

이렇게 GitHub Actions나 서버리스 함수로 **자동화된 무인 파이프라인**을 만들면, 이 건 더 이상 "사람이 로그인해서 구독으로 쓰는" 경로가 아니라 **API 키를 이용한 프로그래밍적 호출**이 됩니다. 즉 이전에 설명드린 "이중 지출" 구분에서 **이 자동화 파이프라인 부분은 구독이 아니라 API 종량제 요금으로 청구되는 영역**에 해당합니다. 자료 등록 빈도, 웹페이지 질의 빈도에 비례해서 이 부분만큼은 별도 API 비용이 실제로 발생한다고 보셔야 합니다.

정리 — 필요한 구성 요소

| 구성 요소                                      | 역할                                        |
|--------------------------------------------|-------------------------------------------|
| GitHub Actions (my_webpage)                | 자료실 업로드 감지 → LLM Wiki 저장소로 파일 전달          |
| GitHub Actions (LLM_Wiki_DataScience)      | 새 raw 파일 감지 → 헤드리스 ingest 실행 → wiki 자동 커밋 |
| Claude Agent SDK 또는 <code>claude -p</code> | 사람 개입 없이 ingest를 수행하는 헤드리스 실행 방식          |
| 서버리스 백엔드 (Cloudflare/Vercel 등)             | 웹페이지 질문을 받아 위키를 읽고 Claude API로 답변 생성      |
| API 키(비밀 환경변수)                             | 자동화 구간에서만 사용, 절대 프론트엔드 코드에 노출 금지          |

**요약하면:** 개념적으로 말씀하신 파이프라인은 충분히 구현 가능하고, 실제로 커뮤니티에서 비슷한 자동화 사례들이 나오고 있습니다. 다만 지금까지의 "대화형 Claude Code 사용"과는 다르게, **GitHub Actions 워크플로우 작성, 헤드리스 실행 스크립트, 서버리스 백엔드**라는 세 가지를 실제로 개발해야 하는 별도 엔지니어링 작업입니다.



이 정도 규모의 실제 구현 작업은 대화창보다 코드를 직접 짜고 실행하며 반복 검증하는 환경이 훨씬 잘 맞습니다.

## Q29. 사용자

---

그럼 이런건 두 프로젝트 중 어떤 프로젝트의 Claude code에 물어보고 진행하는게 좋을까?

## A29. 어시스턴트(Claude)

---

좋은 실무적 질문입니다. 결론부터 말씀드리면 "어느 한쪽에서만 다 해결한다"기보다, 순서와 접근 방식을 나누는 게 낫습니다.

### 왜 한쪽만으로는 어려운가

이 연동 작업은 정의상 두 저장소 사이에 걸쳐 있습니다:

- my\_webpage 쪽에 만들어야 할 것: 자료실 업로드 감지 → LLM Wiki 로 파일 전달하는 GitHub Actions
- LLM\_Wiki\_DataScience 쪽에 만들어야 할 것: raw 파일 수신 → 헤드리스 ingest 실행 → 질의 응답을 처리할 백엔드 로직

한쪽 프로젝트의 Claude Code 세션은 기본적으로 그 폴더 안의 파일만 보고 작업합니다. 그래서 어느 한쪽에서만 작업을 시작하면, 다른 쪽 저장소 구조나 규칙을 모른 채로 "일단 우리 쪽만 완성하고 상대는 나중에" 식이 되기 쉽습니다.

추천 순서 — LLM\_Wiki\_DataScience 부터 시작하시는 게 좋습니다

이유는 이렇습니다:

1. **LLM\_Wiki\_DataScience 가 아직 초기 세팅 전 단계**이기 때문에, 여기서 먼저 "이 위키가 외부로부터 자료를 어떻게 받을 것인지, 질의 요청을 어떤 형태로 받아서 어떻게 답할 것인지"에 대한 **규격(contract)**을 먼저 정의하는 게 순서상 맞습니다.
2. 이 규격이 먼저 나와야, my\_webpage 쪽에서 "그 규격에 맞춰서 파일을 보내는" GitHub Actions 를 제대로 짤 수 있습니다. 반대로 하면 my\_webpage 쪽 로직을 나중에 다시 뜯어고쳐야 할 가능성이 큼니다.

## LLM\_Wiki\_DataScience의 Claude Code에게 먼저 물어볼 것:

- CLAUDE.md 스키마 작성 시 "자동화 파이프라인으로부터 raw 파일을 받는다"는 전제를 포함해서 설계
- 헤드리스(claude -p 등) ingest 를 트리거하는 GitHub Actions 워크플로우 작성
- 질의를 받아 답을 반환하는 백엔드 로직(어떤 입출력 형식을 쓸지)

## 그 다음 my\_webpage의 Claude Code에게 물어볼 것:

- 자료실 업로드 감지 → LLM Wiki 저장소로 파일을 push 하는 GitHub Actions
- LLM Wiki 가 정해진 백엔드 엔드포인트로 질문을 보내고 답을 받아와서 화면에 출력하는 프론트엔드/서버리스 함수 코드

만약 두 저장소를 동시에 다뤄야 하는 부분이 생기면

두 저장소가 로컬 디스크에서 형제 폴더 구조(`~/claude/projects/my_webpage/`, `~/claude/projects/llm-wiki-datascience/` 같은)로 있으시다면, 그 상위 폴더에서 새 Claude Code 세션을 하나 열어서 두 저장소를 한 번에 보이게 하고 "연동 부분만" 작업하는 방법도 있습니다. 이렇게 하면 규격 불일치 문제 없이 한 번에 양쪽 워크플로우 파일을 일관되게 작성할 수 있습니다.

## 정리

| 순서   | 어느 프로젝트 Claude Code   | 하는 일                                          |
|------|-----------------------|-----------------------------------------------|
| 1    | LLM_Wiki_DataScience  | 위키 초기 설정 + 외부 수신/응답 규격 정의 + 헤드리스 ingest 워크플로우 |
| 2    | my_webpage            | 규격에 맞춰 자료 전달 액션 + 질의 요청/답변 표시 로직              |
| (선택) | 두 저장소를 함께 보는 상위 폴더 세션 | 연동 부분(양쪽 워크플로우 파일)을 한 번에 일관되게 작성              |

요약하면, "받는 쪽(LLM Wiki)"에서 규격을 먼저 확정하고, "보내는 쪽(my\_webpage)"이 거기에 맞추는 순서로 진행하시는 걸 추천드립니다.

Q30. 사용자

웹페이지 질문을 받아 위키를 읽고 답변하는 기능이 없다면 API 비용이 발생하지는  
않아?

A30. 어시스턴트(Claude)

좋은 질문입니다. 결론부터 말씀드리면, **질의응답 기능만 뺀다고 API 비용이 0이 되는 건 아닙니다.** 왜냐하면 앞서 설명드린 두 흐름 중 **자료 자동 수집(ingest)** 흐름 자체가 이미 **API 비용이 발생하는 구조**이기 때문입니다.

왜 그런가 — 비용이 발생하는 기준을 다시 짚어보면

이전에 말씀드린 기준은 **"사람이 로그인해서 대화하며 쓰는가" vs "자동화 스크립트가 API 키로 무인 호출하는가"**였습니다.

| 흐름               | 실행 방식                                     | 비용                      |
|------------------|-------------------------------------------|-------------------------|
| 자료 자동 수집(ingest) | GitHub Actions가 사람 없이 자동으로 헤드리스 Claude 호출 | <b>API 키 사용 → 비용 발생</b> |
| 웹페이지 질의응답        | 서버리스 백엔드가 사람 없이 자동으로 Claude API 호출        | <b>API 키 사용 → 비용 발생</b> |

두 흐름 다 **"무인 자동화"**라는 공통점이 있어서, **질의응답만 뺀다고 자료 수집 쪽 비용까지 사라지진 않습니다.** 자료실에 파일을 올릴 때마다 GitHub Actions가 자동으로 Claude를 불러서 ingest를 수행하는 이상, 그 호출 자체가 이미 API 종량제 청구 대상입니다.

그럼 비용이 0 이 되려면 어떻게 해야 하나

정말로 API 비용을 아예 안 내고 싶으시면, **자동화(GitHub Actions 트리거)를 없애고, ingest도 사람이 직접 Claude Code 터미널에서 명령해야 하는 구조로 되돌려야** 합니다. 즉:

자동화 O (지금 설계) → 자료 올리면 자동으로 ingest 실행 → API 비용 발생  
자동화 X (수동 방식) → 자료를 raw/ 폴더에 넣어두고, 사람이 나중에 Claude Code 열어서 "수집해줘" 직접 명령 → 구독 사용량으로 처리, 추가 비용 없음

그러니까 질의응답 기능 유무는 "비용을 얼마나 더 낼지"의 문제이지 "비용이 나느냐 마느냐"의 문제가 아닙니다

정리하면 이렇게 4가지 조합이 나옵니다:

| 자동 ingest | 자동 질의응답        | 비용                                                                        |
|-----------|----------------|---------------------------------------------------------------------------|
| O         | O              | 두 흐름 다 API 비용 발생 (가장 비쌈)                                                  |
| O         | X (지금 질문하신 경우) | ingest 자동화 부분만 API 비용 발생, 질의응답은 없으니 그 부분 비용은 없음 — <b>비용이 줄긴 하지만 0은 아님</b> |
| X         | O              | 질의응답 부분만 API 비용 발생                                                        |
| X         | X              | 전부 사람이 Claude Code로 수동 진행 → 구독 사용량 안에서 해결, 추가 비용 없음                       |

### 실질적으로 얼마나 차이가 날지

두 흐름 중 어느 쪽이 더 비용이 큰지는 **호출 빈도와 처리량**에 달려 있습니다:

- **ingest**: 자료 하나당 여러 페이지를 다시 쓰는 작업이라 1 회당 비용이 큰 편이지만, 자료를 등록하는 빈도는 상대적으로 낮을 수 있음 (예: 하루 몇 건)
- **질의응답**: 1 회당 비용은 ingest 보다 작지만(관련 페이지 몇 개만 읽는 수준), 웹사이트 방문자가 자주 질문할수록 호출 빈도가 훨씬 높아질 수 있음

즉 방문자가 많은 웹사이트라면 질의응답 쪽을 빼는 게 오히려 비용 절감 효과가 더 클 수도 있고, 반대로 자료를 자주 대량으로 올리는 상황이라면 ingest 쪽 비용이 더 클 수도 있습니다. 정확한 판단을 위해서는 실제 예상 사용 빈도를 가지고 따져보시는 게 좋습니다.